

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 唐菁 学号 2024K8009970006 专业 计算机科学与技术
实验项目编号 3 实验名称 定制功能型处理器设计

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 /home/serve-ide/cod-lab/reports 目录下(注意: reports 全部小写)。文件命名规则: prjN.pdf, 其中 prj 和后缀名 pdf 为小写, N 为 1 至 4 的阿拉伯数字。例如: prj1.pdf。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: prj5-projectname.pdf, 其中“-”为英文标点符号的短横线。文件命名举例: prj5-dma.pdf。具体要求详见实验项目 5 讲义。
- 注 2: 使用 git add 及 git commit 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 git push 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

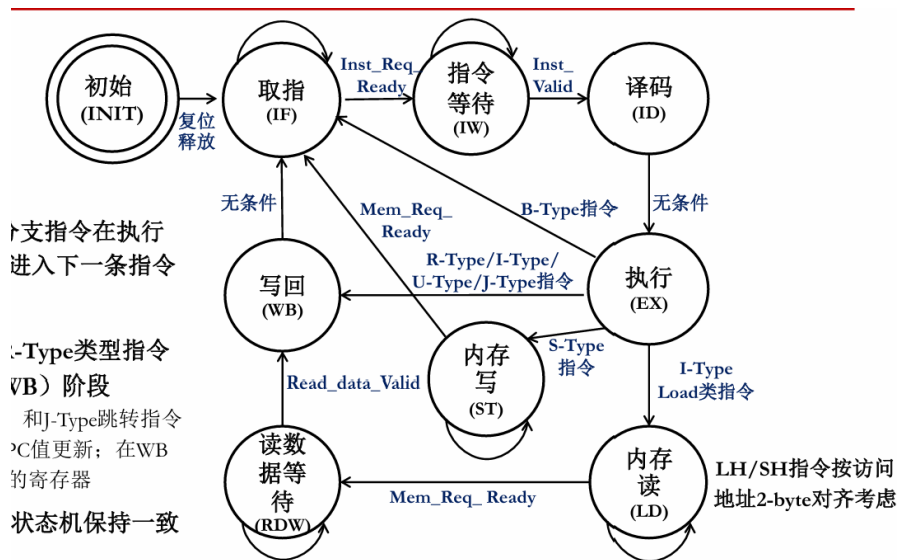
一、基本思路说明、具体设计细节

1 custom_cpu 的设计

1.1 状态机理解

上一个实验已经实现了单周期、多周期处理器的设置。这次将重点从原来的多周期处理器和现在的定制型处理器不同的地方加以阐释。

PPT 上的图画得很好了, 这里直接借用。但是为了表示我的理解, 接下来将进行阐释:



我们主要将一个指令分成这几个部分: 取指、译码、执行、写回、读写。但是, 这次是真实内存, 不能够一个周期一定就可以回来数据; 为了探寻到底数据成功读取/写回没有, 我们增加三个状态和几个握手信号: 指令等待、内存读(为了等待读)、内存写(为了等待写); 以及各种 ready/valid 信号。然后我们再增加初始的 INIT, 因为初始的状态和取指的信号不太一致。

我们完成一个指令,需要做以下这几步。

1. 开头 INIT 的时候,将 Inst_Req_Ready 设成低(防止还没开始取指就开始读指令了);然后将 Inst_ready 和 Inst_Valid 拉高,防止对后面造成影响(比如握手死锁,等等。)

2. 然后进入取指状态,我们发出 Inst_req_Valid,表示发出指令,然后当对面接收到这个获取指令的要求之后,发出 Inst_req_ready,表示接受到了这个指令请求,开始准备指令。当存储器准备好指令之后,就发给我 Inst_valid,表示发给我的指令有效。此时我们终于拿到了指令,进入译码状态。

3. 译码之中,顺便把 branch address 算了

4. 译码完成后,如果是跳转/branch 指令,那么直接将算出来的结果赋值给 PC,回到 IF 阶段

5. 如果执行内存写,那么进入内存写阶段,直到写好了之后,存储体发出 Mem_Req_Ready,表示接受到了写请求(即写完了),那么 STORE 指令完成

6. 如果执行内存读,首先当内存接收到了要求之后(即 Mem_Req_Ready 变高),然后进入读数据等待,直到读到了数据(即 Read_data_Valid 高),这条指令才结束。

1.2 一些功能部件的设计的分析

1. PC

PC 的更新有这几次:

(1) 开头直接算出 PC+4,放进 Result 里面,然后在 IW->ID 的时候直接赋值给 PC。

(2) 如果是 branch 指令并且分支条件符合,那么就在执行阶段确认结果之后,结果在 ALUOUT 寄存器里,直接赋值给 PC。因为这个时候,执行的时候算出来的分支是否采用的结果还在 Result 里,还没有进入 Aluout 覆盖,所以这个时候,将 ALUout 的结果给 PC 是正确的。

(3) 如果是 JAL 等指令,就在执行阶段进行计算。计算的结果如果进入 ALUout,就会慢一拍,所以我采用直接从 ALU 的 Result(组合逻辑结果 output),赋值给 PC。

即:

```
1 assign PCSource = (current_state == `IF || is_J_JAL || current_state == `IW)? `pcfromresult :  
    // 跳转指令/PC+4 来自于 result  
2 `pcfromaluout; // branch 分支指令, 来自于 aluout
```

2. 状态转移机: 是对上面的状态转移的直接刻画,在 always 块里面,根据当前状态、信号和指令类型进行直接的判断。

3. 其他的关于 ALU 的计算数字的选择、各个控制信号,下面进行简要阐释。

(1) 仍然进行类型判断: 拒绝直接通过码字每一次进行判断,而是通过码字-> 类型-> 进行判断,这样抽象一层会更好。然后,根据类型,立即数也可以立即判断出来。

```
1 assign is_R    = (opcode == 7'b0110011);  
2 assign is_I    = (opcode == 7'b0010011); // 算术逻辑 I 型  
3 assign is_L    = (opcode == 7'b0000011); // Load  
4 assign is_S    = (opcode == 7'b0100011); // Store  
5 assign is_B    = (opcode == 7'b1100011); // Branch  
6 assign is_U_LUI = (opcode == 7'b0110111);  
7 assign is_U_AUIPC = (opcode == 7'b0010111);  
8 assign is_J_JAL = (opcode == 7'b1101111);  
9 assign is_J_JALR = (opcode == 7'b1100111);
```

(2) ALU 的使用(复用...)

a. IW 的时候算 PC+4

b. branch 指令在 ID 阶段算地址

c. branch 指令在执行阶段算是否小于(有符号、无符号)、是否相等(相减,看 Zero)

d. 各个算术指令的对应计算类型,在执行的时候。

e.load/store/auipc 算地址,在 EX 阶段。

对应下来的 ALUOP 如下,然后再根据上面说的,将 A 和 B 包含哪些总结一下,生成 AlusrcA 和 AlusrcB 这两个控制信号,用这些信号来控制 A 和 B,那么操作数也就做完了。

```
1 assign ALUop =
2 (current_state == `IF || current_state == `IW)? `ADD: //算PC
3 (is_B && ((funct3 == 3'b110) || (funct3 == 3'b111)) && (current_state == `EX)? `SLTU :// BGEU/BEU
4 (is_B && (funct3 == 3'b100 || funct3 == 3'b101)) && (current_state == `EX)? `SLT :// BLT, BGE
5 (is_L || is_S || is_U_AUIPC)? `ADD : // 访存地址计算 / AUIPC
6 (is_B && (current_state == `ID)) ? `ADD : //算地址
7 (is_B && (current_state == `EX)) ? `SUB : //算地址
8 ((is_R || is_I) && funct3 == 3'b000) ? ((is_R && Ins[30]) ? `SUB : `ADD) : // add/sub
   (仅R型有sub)
9 ((is_R || is_I) && funct3 == 3'b010) ? `SLT :// slt
10 ((is_R || is_I) && funct3 == 3'b011) ? `SLTU :// sltu
11 ((is_R || is_I) && funct3 == 3'b100) ? `XOR :// xor
12 ((is_R || is_I) && funct3 == 3'b110) ? `OR :// or
13 ((is_R || is_I) && funct3 == 3'b111) ? `AND :// and
14 (is_J_JAL || is_J_JALR)? `ADD:
15 `ADD; // default, 这样平时PC+4也会合理
```

(3) Memory 的控制信号的发出(掩码信号等略)

因为这次加了读写等待状态(因为真实内存),所以控制信号略有改变:

```
1 assign MemWrite = (current_state == `ST) && (is_S);
2 assign MemRead = ((current_state == `LD) && (is_L));
3 assign MentoReg = is_L;
```

这次的读写主要发生在 ST 和 LD 阶段,这次也和上次的多周期不同,是分开两个状态(因为握手信号不一样了)

1 性能计数器的各层实现

1.1 custom cpu: 底层实现

想了想,实现了:周期计数(每个周期开始的时候加 1)、指令数计数(每次跳到 IF 都加 1)、几个等待周期打转的时候加 1、load/store 两种指令每次遇见了加 1,实现代码是:

```
1 else begin
2   cpu_perf_cnt_0 <= cpu_perf_cnt_0 + 1; //周期数+1
3   case (current_state)
4     `INIT:begin
5       current_state <= `IF;
6     end
7     `IF: begin
8       if (Inst_Req_Ready)begin
9         current_state <= `IW;
10      end
```

```

11  else begin
12  cpu_perf_cnt_2 <= cpu_perf_cnt_2 + 1; //取指原地打转
13  end
14  end
15
16  `IW: begin
17  if (Inst_Valid)begin
18  current_state <= `ID;
19  end
20  else begin
21  cpu_perf_cnt_8 <= cpu_perf_cnt_8 + 1; //IW原地打转
22  end
23  end
24
25  `ID:begin
26
27  current_state <= `EX;
28
29
30  end
31
32  `EX: begin
33
34  if (is_L ) begin
35  current_state <= `LD;
36  cpu_perf_cnt_6 <= cpu_perf_cnt_6 + 1; //load指令+1
37  end
38
39  else if (is_R || is_I || is_U_AUIPC || is_U_LUI || is_J_JAL || is_J_JALR) begin
40  current_state <= `WB;
41  end
42
43  else if (is_B) begin
44  current_state <= `IF;
45  cpu_perf_cnt_1 <= cpu_perf_cnt_1 + 1; //指令数+1
46  end
47  else if (is_S) begin
48  current_state <=`ST;
49  cpu_perf_cnt_7 <= cpu_perf_cnt_7 + 1; //store指令+1
50  end
51  else begin
52  current_state <= `IF;
53  end
54  end
55
56  `ST: begin
57  if((Mem_Req_Ready))begin
58  current_state <= `IF;
59  cpu_perf_cnt_1 <= cpu_perf_cnt_1 +1;

```

```

60     end
61     else begin
62         cpu_perf_cnt_3 <= cpu_perf_cnt_3 + 1; //写原地打转
63     end
64 end
65
66 `LD: begin
67     if(Mem_Req_Ready)begin
68         current_state <= `RDW;
69     end
70     else begin
71         cpu_perf_cnt_4 <= cpu_perf_cnt_4 + 1; //load 原地打转
72     end
73 end
74 `WB: begin
75
76     current_state <= `IF;
77     cpu_perf_cnt_1 <= cpu_perf_cnt_1 + 1;
78 end
79 `RDW:begin
80     if(Read_data_Valid)begin
81         current_state <= `WB;
82     end
83     else begin
84         cpu_perf_cnt_5 <= cpu_perf_cnt_5 + 1; //read data waiting
85     end
86 end
87 default: begin
88     current_state <= `IF;
89 end
90 endcase
91
92 end

```

1.2 perf_cnt:C 代码,实现 wheel 函数来对底层的计数器进行读;perf_cnt.h: 实现 Result 结构体来记录计数器的结果

1. 先实现访问函数:这些函数返回对应的计数器的数值。这个数值怎么计算呢? 是基址,加上偏移量。但是偏移量因为是 int 指针,指针 +1 加的是指针的大小,int 是 4 个字节,所以应该是基址加上偏移量除以 4,这样才是访问的对应的计数器。

for example:

```

1 void bench_prepare(Result *res) {
2     res->msec = _uptime();
3     res->insnum = _insnum();
4     res->ifcycle = _IF_cycle();//其余略
5 }

```

2. 再实现获取状态函数:再 bench_prepare 中记录下当前计数器的数量,并且存进 Result 结构体里(因此 Result

结构体里也应该进行相应的添加); bench_done 则是将当前计数器的结果减去刚刚存下来的结果, 得到这段时间中的改变量。

for example:

```
1 void bench_done(Result *res) {
2     res->msec    = _uptime() - res->msec;
3     res->insnum  = _insnum() - res->insnum; //其余略
```

1.3 bench.c: 真正进行测试, 打印结果(高层实现)

每一次跑 run_once 的时候, 都会使用 Perf_cnt.c 中的 bench_prepare 和 bench_done, 将结果存在 result 里。然后, 进行打印:

```
1 printk("The first one is the cycle-nums:%u\n",res.msec);
2 printk("The second one is the ins num:%u\n",res.insnum);
3 printk("The third one is in the IF waiting to be in IW:%u\n",res.ifcycle); //其余略
```

1 printf 中 put 函数的实现

逻辑如 PPT, 已然十分清晰了:

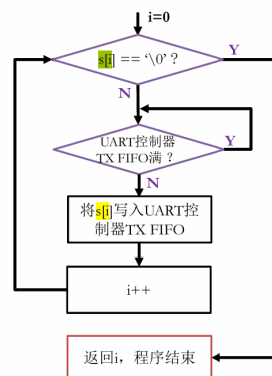


图 2: printf 中 put 函数的实现

也就是打印这个字符串。如果控制器没有满, 就在对应位置写入控制器, 如果满了, 就进行等待。

其中, 控制器的位置是 uart 基址加上 uart_tx_fifo 的偏移量。即:

```
1 *(uart+UART_TX_FIFO/4) = s[i];
```

而如何判断是否满了呢? 我们是通过 STAT_REG 寄存器的第 3 比特(最低位为第 0 比特)是否为 1 来看发送队列到达满状态。那么我们只需要让 uart 加上 status 的偏移量, 拿到 stat_reg 的地址(同样, 偏移量还是要除以 4), 然后与上 1000, 这样可以取出第 3 位, 拿到是否满的标记。如果满了, 就 continue, 进行等待。

即:

```
1 while (((*(uart + UART_STATUS/4)) & (UART_TX_FIFO_FULL))){
2     continue;
3 }
```

二、 实验中遇到的 Bug 等问题的解决和思考

2 custom_cpu 的问题

1. PC 的问题

因为是直接的多周期上改的,PCwrite 就仍然写的是在 IF 的时候更新,这是非常错的。

首先,如果在 IF 更新,那么下一拍 IW 拿到的就是下一条指令;其次,如果在 IW 不分雌雄地更新,如果 IW 有多个周期,每一个上升沿都更新一次,那么就会不止加 4;所以,更新要求是 IW->ID,即

```
(current_state == `IW) && (Inst_Valid)
```

2. 到底结果来自 ALUOUT 这个寄存器,还是来自于 Result 这个直接组合逻辑的输出

如果是 branch 指令,只能来自于 Aluout,因为在执行阶段计算 branch_taken(分支条件) 的时候,Result 已经被覆盖了。

而 PC+4 的更新来自于 Result 会比较好。万一 IW 只有一拍,PC+4 还没来得及从 Result 存进 Aluout,就已经要更新 PC 了。

跳转指令 (JAL) 也是这样,万一执行只有一拍,算出来的结果还没来得及从 Result 存进 Aluout,就已经要更新 PC 了。

(因为其实是在上一次多周期之上改的,所以没有太多 bug)

2 性能计数器的问题

1. 最开始没有搞清楚,perf_cnt.c 这个文件是通过地址实现对计数器的访问,将访问抽象成函数;而 bench.c 直接用这个函数就行了。所以最开始在 bench.c 里面努力写地址访问,然后很困惑 perf_cnt.c 是干什么的,以及怎么实现对计数器的结果在开始和结尾的记录。后来读了一下 run_once 函数中的 bench_prepare 和 bench_done 函数,才发现基本的访问是在 perf_cnt.c 文件里实现的,bench 里面直接用就行了,才明白文件彼此的层次结构关系

2. 以为对地址解引用只要是个数字就可以解引用,所以将 cnt 的地址定义为普通整数,出了问题。只能对指针解引用。而正由于是(整型)指针,所以 +1 的意思就是 +4 个字节

2 printf 的问题

当时判断是否满了的时候,最开始写的是:

```
while((*uart + UART_STATUS/4) & (UART_TX_FIFO_FULL) == 1)
```

然后报 warning,提醒我按位与的优先度低于 ==,应该把前面按位与的结果括起来。

三、 对讲义中思考题(如有)的理解和回答

1. 处理器内部是否需要实现异步串行通信协议?

答:完全不需要。原因如下:

(1) 处理器内部互连需要的是“将数据尽可能快、尽可能宽、与时钟精确同步地搬移”,而 UART 协议是为“没有公共时钟、距离远、速率低”的场景设计的。

(2)UART 协议开销极大,每发送 5 9 位数据,至少需要 1 个起始位和 1 个停止位。并且速度无法满足内部需求:UART 的典型速率在 kbps 几 Mbps,即使极限也仅几十 Mbps。内部应该用总线等方式。

(3)UART 帧组装/拆解有天生延迟:发送方要等待整个字符帧按位发出,接收方要等停止位结束才能提取数据。

(4) 内部已是全局同步/源同步体系:处理器内部所有模块共享经过精心设计的时钟树,不需要异步了。

2. 处理器如何访问 UART 控制器的 I/O 端口?

答:本实验采用统一编址方式,I/O 端口访问可复用内存访问通路。所以 C 语言访问的时候直接写出地址解引用即可。

3. 下图中 volatile 关键字的作用是什么? 如果去掉会出现什么后果?

| UART控制器寄存器接口定义 (benchmark/common/printf.c)

```
#define UART_TX_FIFO      0x04
#define UART_STATUS      0x08
#define UART_TX_FIFO_FULL (1 << 3)

volatile unsigned int *uart = (void *)0x60000000;
```

UART发送数据队列入口寄存器偏移地址
UART队列状态寄存器偏移地址
UART发送数据队列状态标志位掩码
UART控制器基地址指针 (地址不可修)

图 3: volatile 关键字的作用

防止编译器强行进行优化,后续进行比对的时候,将 `uart+UART_STATUS/4` 中的第一次的数据存起来放在寄存器里,后面就不去读这个地址对应的内容了,而是直接从寄存器里去看。这样有可能后面其实是满的,但是仍然认为没有满,然后将字符串放进串口,就会丢失打印;或者没有满,但是一直卡在满的状态,进入死循环。

不过我将 `volatile` 删去,和正确结果倒是一样的。可能串口状态一直没有满过。

```
1 while (((*(uart + UART_STATUS/4))&(UART_TX_FIFO_FULL) )){
2     continue;
3 }
```

四、 实验所耗时间

在课后,你花费了大约 3 小时完成此次实验。